POST-QUANTUM CRYPTOGRAPHY

# CRYSTALS-Dilithium

Sourabh Singh & Utkarsh Mishra

Quantum-Resistant

Lattice-Based

# Learning Path

Seven structured modules covering the foundations of CRYSTALS-Dilithium

**01** **Post-Quantum Cryptography**

Understanding the need for next-gen cryptography

**02** **Quantum Threat & Complexity**

How quantum computers break classical security

**03** **Hard Problems in Cryptography**

Mathematical hardness assumptions

**04** **Lattice Foundations**

Geometric basis for new hard problems

**05** **Learning With Errors (LWE)**

The core hardness assumption

**06** **Module LWE (MLWE)**

Polynomial extension for efficiency

**07** **Connection to Dilithium**

From theory to real-world application

MODULE 01

# Post-Quantum Cryptography

Understanding the need for next-generation cryptography

MODULE 01

# Post-Quantum Cryptography

*Cryptography designed to remain secure against attacks from both classical and quantum computers*

## Purpose

- Replaces RSA and ECC
- Secure post-quantum era
- NIST standardized (2024)

## Mathematical Basis

- Lattice-based problems
- Code-based problems
- Hash-based schemes

## Key Properties

- Quantum-resistant
- Efficient computation
- Compact key sizes

MODULE 01

# Quantum Threat & Complexity

## Classical Cryptography

**RSA** Integer factorization

**ECC** Discrete logarithm

**Attack** Sub-exponential time

**Status** Secure classically

## Quantum Computing Threat

**Shor's Algorithm** Breaks RSA & ECC

**Complexity** $O(n^3)$ polynomial time

**RSA-2048** Broken in hours*

**Timeline** 2029–2035 (est.)

**Classical → billions of years | Quantum → hours (theoretical)***

MODULE 01

# Hard Problems in Cryptography

Security relies on computational hardness — problems easy to set up but hard to solve

**01** **Integer Factorization**

*Given $n = p \times q$, find primes $p$ and $q$*

Basis of RSA — hard classically, broken by Shor's

**02** **Discrete Logarithm**

*Given $g$, $g^x \bmod p$, find $x$*

Basis of ECC/Diffie-Hellman — broken by Shor's

**03** **Shortest Vector Problem (SVP)**

*Find shortest non-zero vector in a lattice*

Basis of lattice crypto — quantum-resistant

**04** **Learning With Errors (LWE)**

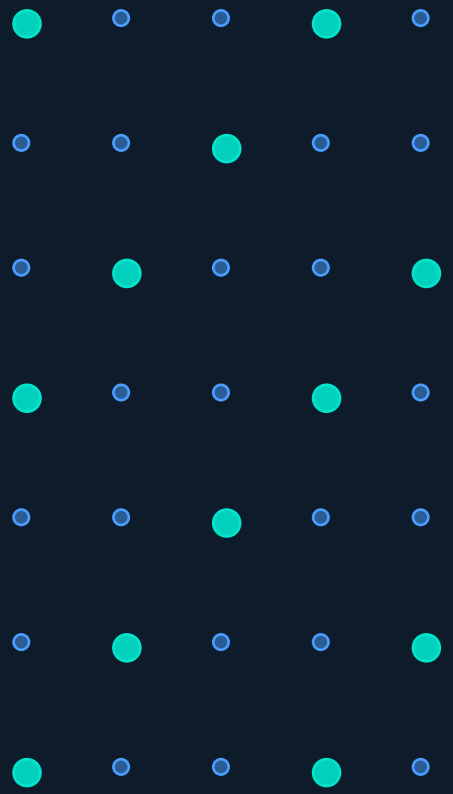
*Recover $s$ from noisy equations $A \cdot s + e$*

Post-quantum hard problem — powers Dilithium

MODULE 02

# Lattice Foundations

The geometric basis for quantum-resistant cryptography



MODULE 02

# What is a Lattice?

$$L(b_1, b_2, \ldots, b_n) = \{ \sum a_i b_i \mid a_i \in \mathbb{Z} \}$$

### Grid Structure

A discrete set of points in n-dimensional space

### Basis Vectors

Generated by linearly independent vectors $b_1, b_2, \ldots, b_n$

### Integer Coefficients

All integer combinations of basis vectors

### Foundation

Basis for new computational hard problems in crypto

MODULE 02

# Lattice-Based Cryptography

## Hard Problem: SVP

- Shortest Vector Problem (SVP)
- Find shortest non-zero lattice vector
- NP-hard in general case
- No efficient quantum algorithm known
- Basis of lattice cryptography

## Why Lattices?

### Quantum-Resistant

No known quantum speedup for SVP

### Efficient

Fast key generation and signing

### Scalable

Practical key and signature sizes

### Provable Security

Worst-case to average-case reductions

MODULE 03

# Learning With Errors (LWE)

The core computational hardness assumption

$$y = A \cdot s + e$$

MODULE 03

# The LWE Equation

$$y = A \cdot s + e$$

**A** — **Public Matrix**

Known to everyone — serves as the public key material

**s** — **Secret Vector**

Private key — what an attacker wants to recover

**e** — **Error / Noise**

Small random noise sampled from Gaussian distribution

**y** — **Noisy Output**

Public result — computationally hard to invert without s

**Key Idea:** Noise hides the secret — finding s from y and A is computationally infeasible

# LWE — Matrix Example

$$\underbrace{\begin{bmatrix} 2 & 3 \\ 1 & 4 \\ 3 & 5 \end{bmatrix}}_{A} \times \underbrace{\begin{bmatrix} 1 \\ 2 \end{bmatrix}}_{s} + \underbrace{\begin{bmatrix} 1 \\ 0 \\ -1 \end{bmatrix}}_{e} = \underbrace{\begin{bmatrix} 9 \\ 9 \\ 12 \end{bmatrix}}_{y}$$

**Verification:**

Row 1: $2\times1 + 3\times2 + 1(\text{noise}) = 2 + 6 + 1 = 9$ ✓

Row 2: $1\times1 + 4\times2 + 0(\text{noise}) = 1 + 8 + 0 = 9$ ✓

Row 3: $3\times1 + 5\times2 + (-1)(\text{noise}) = 3 + 10 - 1 = 12$ ✓

# Why LWE is Hard

01

## No Exact Solution

The noise vector e prevents exact algebraic recovery of s. No inverse formula exists.

02

## Noise Creates Ambiguity

Many different (A, s) pairs could produce the observed y — the solution space is enormous.

03

## Related to Lattice Problems

LWE reduces to SVP/CVP which are NP-hard. Breaking LWE would break all lattice crypto.

04

## Quantum-Resistant

No known quantum algorithm provides significant speedup over classical attacks on LWE.

MODULE 04

# Module Learning With Errors (MLWE)

Extending LWE with polynomial rings for efficiency

$$y(x) = A(x) \cdot s(x) + e(x)$$

# MLWE — Core Concept

$$y(x) = A(x) \cdot s(x) + e(x)$$

**A(x)** Matrix of polynomials (not just integers)

**s(x)** Secret is a polynomial vector

**e(x)** Noise is polynomial-valued

**y(x)** Output polynomial — hard to invert

**Extension of LWE:** Polynomials replace integer vectors, enabling structured operations and compact representations

# MLWE — Polynomial Example

**A(x) =**

$3 + 2x + x^2 + 4x^3$

Public matrix polynomial

**s(x) =**

$1 + 3x + 2x^2$

Secret polynomial (unknown)

**e(x) =**

$0 + 1 \cdot x + 0 \cdot x^2$

Small noise polynomial

$$y(x) = A(x) \cdot s(x) + e(x) = (3 + 2x + x^2 + 4x^3)(1 + 3x + 2x^2) + \text{noise}$$

**Expanded (mod $x^k + 1$):**

$y(x) = 3 + 11x + 14x^2 + 12x^3 + \ldots + e(x)$ — coefficients are public, s(x) is hidden

⚠️ **Hard to recover s(x):** Polynomial multiplication hides secret even more than scalar multiplication

# MLWE vs LWE — Comparison

## LWE

**Structure:**

Integer vectors and matrices

**Matrix A:**

m×n integer matrix

**Key Size:**

Large — $O(n^2)$ integers

**Speed:**

Slower operations

**Security:**

Proven hard (NP-hard base)

## MLWE ✦ Preferred

**Structure:**

Polynomials in ring R_q

**Matrix A:**

k×l matrix of polynomials

**Key Size:**

Compact — O(n log n)

**Speed:**

Faster via NTT

**Security:**

Reduces to worst-case LWE

MODULE 04

# From LWE ⟶ MLWE ⟶ Dilithium

## 01

### LWE

Theoretical Hardness

- Core hardness assumption
- Proved quantum-resistant
- Too slow for practice

## 02

### MLWE

Efficiency Layer

- Structured polynomial rings
- Fast NTT computation
- Compact key sizes

## 03

### Dilithium

Real-World Application

- NIST PQC Standard (2024)
- Digital signature scheme
- Quantum-secure deployable

Each layer builds on the previous: hardness ⟶ efficiency ⟶ practical deployment

# Covered Till Now…

Key Takeaways

**01** **Quantum computers break classical crypto**

Shor's algorithm defeats RSA and ECC in polynomial time

**02** **Lattices provide the solution**

Geometric hard problems (SVP/CVP) remain hard even for quantum computers

**03** **LWE = Core hardness**

Noisy linear equations over integers that no known algorithm can solve efficiently

**04** **MLWE = Efficiency**

Polynomial rings enable compact keys and fast operations via NTT

**05** **Leads to Dilithium**

NIST-standardized (2024) post-quantum digital signature — production-ready

POST-QUANTUM CRYPTOGRAPHY

# CRYSTALS-Dilithium Working

A Post-Quantum Digital Signature Scheme

April 2026

# Learning Path

Eight comprehensive modules covering all aspects of CRYSTALS-Dilithium

### 01 Introduction to Post-Quantum Cryptography

Understanding the quantum threat, why traditional cryptography fails, and why CRYSTALS-Dilithium is essential for the quantum era

### 02 Mathematical Building Blocks

Polynomial rings, parameter sets, vectors and matrices of polynomials—the algebraic foundation of lattice-based cryptography

### 03 Key Generation with Examples

Step-by-step key generation: seed expansion, matrix creation, secret sampling, and public key derivation with numerical examples

### 04 Signing Process with Examples

Complete signing workflow: message hashing, masking, challenge creation, rejection sampling, and signature output with detailed steps

### 05 Verification Process with Examples

Signature validation: matrix regeneration, challenge reconstruction, hint application, and acceptance criteria with mathematical proofs

### 06 Complete Worked Example

End-to-end demonstration with toy parameters: key generation, signing "Hello", and verification with simplified arithmetic

### 07 Security Analysis

MLWE and MSIS hardness assumptions, quantum resistance, rejection sampling importance, and attack prevention mechanisms

### 08 Summary and Key Takeaways

Comprehensive recap of all operations, Dilithium's unique advantages, and why it represents the future of digital signatures

MODULE 01

# CRYSTALS-Dilithium: Key Features and Advantages

## Security Basis

**Module Learning With Errors (MLWE)** and **Module Short Integer Solution (MSIS)** problems

- ✓ Resistant to both classical and quantum attacks
  ~125 bits of quantum security

## Implementation

**Easy to implement in constant-time** without branches, making it resistant to timing attacks

- ✓ No secret-dependent branches
  Side-channel resistant by design

## Sampling

**Uses uniform random sampling**—no complex Gaussian sampling required, unlike other lattice schemes

- ✓ Simpler implementation
  Faster key generation

**~1.5 KB**
Public Key Size
Compact for lattice-based scheme

**~2.7 KB**
Signature Size
Moderate, smaller than many PQC schemes

**Fast**
Signing & Verification
Optimized with NTT for polynomial multiplication

**NIST**
Standardized
Selected as primary digital signature standard

MODULE 02

# Mathematical Building Blocks

The algebraic foundation: polynomial rings, vectors, and matrices

MODULE 02

# Dilithium Parameters and Their Meanings

## Complete Parameter Set

| Parameter | Value | Meaning |
|---|---|---|
| $q$ | 8,380,417 | Prime modulus ($2^{23} - 2^{13} + 1$) |
| $n$ | 255 | Polynomial degree |
| $k$ | 5 | Matrix A rows |
| $\ell$ | 4 | Matrix A columns |
| $\eta$ | 5 | Secret key coefficient bound |
| $\gamma_1$ | 523,776 | Masking vector bound |
| $\gamma_2$ | 261,888 | Splitting parameter ($\gamma_1/2$) |
| $d$ | 14 | Bits dropped from public key |
| $\omega$ | 60 | Challenge weight (±1 count) |

## Key Design Choices

**$q = 2^{23} - 2^{13} + 1$:** Specially chosen prime enabling fast NTT (Number Theoretic Transform) for polynomial multiplication

**$n = 256$:** Powers of 2 allow efficient polynomial operations via divide-and-conquer algorithms

**$k=5, \ell=4$:** Matrix dimensions balancing security and performance

**$\eta = 5$:** Small secret coefficients make MLWE problem hard while keeping operations efficient

## Security Parameters

**$\gamma_1 = 523{,}776$:** Masking vector range ensures z hides secret key $s_1$

**$\gamma_2 = 261{,}888$:** Half of $\gamma_1$, used for bit splitting and hint generation

**$d = 14$:** Dropping 14 bits compresses public key from ~3.8KB to ~1.5KB

**$\omega = 60$:** Challenge polynomial has exactly 60 non-zero coefficients (±1)

MODULE 02

# Polynomial Ring R_q

## Ring Definition

$$R_q = \mathbb{Z}_q[X] / (X^{256} + 1)$$

All polynomial arithmetic in Dilithium occurs in this ring

1. **Polynomials have degree up to 255**—exactly 256 coefficients
2. **Coefficients are integers modulo q** = 8,380,417, in range [0, q-1]
3. **Reduction rule:** $X_{256} = -1$, so $X_{256} \equiv -1 \pmod q$

## Example Polynomial

$$p(X) = 5 + 3X + 0X^2 + 7X^3 + \ldots + 2X_{255}$$

Coefficient vector: [5, 3, 0, 7, ..., 2]

- 256 coefficients total
- Each coefficient in [0, 8,380,416]
- Can also use centered representation: [-(q-1)/2, (q-1)/2]

## Polynomial Operations

**Addition:** Coefficient-wise addition mod q

**Multiplication:** Convolution with $X_{256} = -1$ reduction

**NTT:** Fast multiplication in O(n log n) time

## Why This Ring Structure?

The ring $\mathbb{Z}_q[X]/(X_{256}+1)$ is chosen because **$X_{256}+1$ is a cyclotomic polynomial** that enables the Number Theoretic Transform (NTT), allowing polynomial multiplication in **O(n log n)** instead of $O(n^2)$. The special form of $q = 2^{23}-2^{13}+1$ ensures 256-th roots of unity exist, making NTT efficient.

MODULE 02

# Vectors and Matrices of Polynomials

## → Vectors

A **vector** contains multiple polynomials from R_q. Each element is a complete polynomial with 256 coefficients.

Example: $s_1$ is a vector of $\ell$=4 polynomials

$s_1 = [s_1[0], s_1[1], s_1[2], s_1[3]]$

Each $s_1[i]$ is a polynomial in R_q

## ▦ Matrices

A **matrix** contains polynomials arranged in rows and columns. Matrix A is the public parameter.

Matrix A is $k \times \ell$ = 5×4

A = 5×4 matrix

Contains 20 polynomials total

## ✕ Multiplication

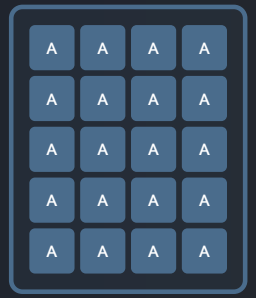
**Matrix-vector multiplication** $A \cdot s_1$ produces a vector of k=5 polynomials.

Result dimension

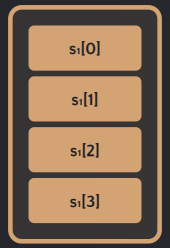
$A \cdot s_1$ = vector of 5 polynomials

Each: polynomial multiplication in R_q

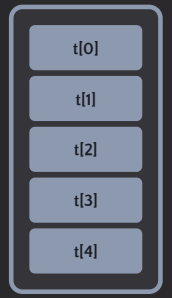
## Visual Representation




Matrix A (5×4)
20 polynomials




Vector $s_1$ (4×1)
4 polynomials




Result t (5×1)
5 polynomials

MODULE 03

# Key Generation

Creating public-private key pairs with detailed step-by-step process

MODULE 03

# Key Generation: Steps 1-3 (Seed, Matrix, Secrets)

## 1 Generate Seed ρ

Generate a **random 256-bit seed** ρ (rho) that will deterministically generate matrix A.

Example:

ρ = 0x3A7F...B2C1

32 bytes (256 bits) in hexadecimal

**Purpose:** Seed for deterministic matrix generation

## 2 Generate Matrix A

Use **SHAKE-128** (extendable output function) to generate A from seed ρ.

Process:

For each position A[i][j]:

1. Feed SHAKE-128: ρ || i || j
2. Extract random bytes
3. Rejection sampling for 256 uniform coefficients
4. Form polynomial A[i][j]

**Result:** 5×4 matrix of polynomials in R_q

## 3 Sample Secrets

Sample **small random polynomials** for secret key vectors $s_1$ and $s_2$.

Secret vectors:

**$s_1$:** Vector of ℓ=4 polynomials, coefficients in [-5, 5]

**$s_2$:** Vector of k=5 polynomials, coefficients in [-5, 5]

Each coefficient randomly chosen from:

{-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5}

**Key property:** All coefficients are SMALL (≤5)

## Example: First Polynomial in $s_1$

$s_1[0] = 2 - 3X + 5X^2 + 0X^3 - 1X^4 + \ldots + 4X^{255}$

Coefficient vector: [2, -3, 5, 0, -1, ..., 4]

**11**
Possible values per coefficient

**256**
Coefficients per polynomial

**9**
Total secret polynomials

MODULE 03

# Key Generation: Steps 4-5 (Compute t and Split)

## 4 Compute $t = A \cdot s_1 + s_2$

Multiply matrix A by vector $s_1$ and add $s_2$. This is the **core computation** in key generation.

Matrix multiplication:

$A \cdot s_1 \rightarrow$ vector of k=5 polynomials

Then add $s_2$ (also 5 polynomials)

Result: t is a vector of 5 polynomials

All arithmetic in R_q (mod q)

**Operation:** Polynomial multiplication + addition in R_q

## 5 Split t into $t_1$ and $t_0$

Split each coefficient of t to **reduce public key size** using Power2Round with d=14.

For each coefficient $t_i$:

$t_i = t_{1,i} \cdot 2^{14} + t_{0,i}$

where $2^{14} = 16{,}384$

$t_{1,i}$: High bits → PUBLIC KEY (9 bits/coeff)

$t_{0,i}$: Low bits → SECRET KEY (14 bits/coeff)

**Savings:** Public key shrinks from ~3.8KB to ~1.5KB

## Numerical Example: Splitting $t_i = 5{,}432{,}100$

**Step 1:** Compute centered reduction modulo $2^{14}$

$5{,}432{,}100 \bmod_{\pm} 16{,}384 = -2{,}948$

Centered in range (-8,192, 8,192]

**Step 2:** Compute high bits

$t_{1,i} = (5{,}432{,}100 - (-2{,}948)) / 16{,}384 = 332$

$t_{0,i} = -2{,}948$ (stored in secret key)

**Verification:** $332 \times 16{,}384 + (-2{,}948) = 5{,}432{,}100$ ✓

MODULE 03

# Key Generation: Final Keys and Sizes

## Public Key (pk)

$$pk = (\rho, t_1)$$

**ρ:** 256-bit seed (32 bytes)
Used to regenerate matrix A

**$t_1$:** High bits of t (~1,440 bytes)
5 polynomials × 256 coeff × 9 bits

**Total Size:** **~1,472 bytes**
Compact for post-quantum scheme

## Secret Key (sk)

$$sk = (\rho, K, tr, s_1, s_2, t_0)$$

- **ρ:** 256-bit seed (32 bytes)
- **K:** 256-bit secret seed for signing (32 bytes)
- **tr:** Hash of public key (48 bytes)
- **$s_1$:** 4 polynomials with small coefficients
- **$s_2$:** 5 polynomials with small coefficients
- **$t_0$:** 5 polynomials with 14-bit coefficients

**Total:** ~2,528 bytes (larger but kept secret)

### Space Optimization Insight

By storing only **high bits $t_1$** in the public key and keeping **low bits $t_0$** secret, Dilithium achieves a **2.5× compression** of the public key. The verifier can still verify signatures using hints (explained in the signing process), making this a clever trade-off between space and verification complexity.

MODULE 04

# Signing Process

Creating digital signatures that prove knowledge of the secret key

MODULE 04

# Signing: Steps 1-3 (Hash, Mask, Compute w)

## 1 Hash Message

Compute a **digest of message M** using collision-resistant hash.

Computation:

m = CRH(tr || M)

tr: hash of public key (from sk)
M: message to sign
CRH: SHAKE-256 producing 384 bits

Example:

M = "Transfer $100 from Alice to Bob"
m = 0x7A3F...9E21 (48 bytes)

**Why hash first?** Signing may retry multiple times

## 2 Generate Mask y

Generate **random masking vector y** to hide the secret key in the signature.

Generation:

y = ExpandMask(K || m || κ)

K: secret seed from sk
m: message digest
κ: counter (starts at 0)
Uses SHAKE-256 to generate $\ell$=4 polynomials

Coefficient bounds:

Range: $[-(\gamma_1-1), \gamma_1-1]$
= [-523,775, 523,775]

**Key insight:** y coefficients (-500K) hide $s_1$ (≤5)

## 3 Compute w = A·y

Multiply the **public matrix A** by masking vector y to create commitment.

Computation:

w = A·y

A: 5×4 matrix of polynomials
y: 4×1 vector of polynomials
Result: w is 5×1 vector
All mod q in R_q

Example for w[0]:

w[0] = A[0][0]·y[0] + A[0][1]·y[1]
+ A[0][2]·y[2] + A[0][3]·y[3]

**Result:** Vector of 5 polynomials in [0, q-1]

## The Masking Principle

The masking vector y is **critical for security**. Since y coefficients are ~500,000 and secret key $s_1$ coefficients are ≤5, the final signature $z = y + c \cdot s_1$ will be dominated by y. This ensures that **z appears random** and doesn't reveal information about the secret key, even to sophisticated attackers.

MODULE 04

# Signing: Steps 4-6 (Extract Bits, Challenge, Compute z)

## 4 Extract $w_1$

Extract **high-order bits** from w for the challenge generation.

Computation:

$w_1 = \text{HighBits}(w, 2\gamma_2)$

where $2\gamma_2 = 523{,}776$

Decompose: $w_i = w_{1,i} \cdot 2\gamma_2 + w_{0,i}$

$w_{0,i}$ in $(-\gamma_2, \gamma_2] = (-261{,}888, 261{,}888]$

Example:

$w_i = 3{,}456{,}789$
$3{,}456{,}789 \div 523{,}776 = 6$ rem $313{,}533$
Center: $313{,}533 - 523{,}776 = -210{,}243$
$w_{1,i} = 7$, $w_{0,i} = -210{,}243$

**Result:** $w_1$ has only 4 bits per coefficient

## 5 Create Challenge c

Hash message digest and $w_1$ to create **challenge polynomial**.

Computation:

$c = H(m \,||\, w_1)$

H: SHAKE-256 hash function

Result: Polynomial in $B_\omega$

Properties of c:

- ✓ Exactly 60 non-zero coefficients
- Non-zero values are ±1
- $L_\infty$ norm = 1
- $L_1$ norm = 60

**Binds signature** to specific message

## 6 Compute z

Compute **signature candidate** combining mask and secret key.

Computation:

$z = y + c \cdot s_1$

y: masking vector (large coefficients)

c: challenge (60 ±1s)

$s_1$: secret key (small coefficients)

Example:

$y_i = 234{,}561$, $c_i = 1$, $s_{1,i} = -3$
$z_i = 234{,}561 + (1)(-3) = 234{,}558$
Change: only -3 (very small!)

**Max impact:** 60 × 5 = 300 (very small)

## Security Insight

Since c has only 60 non-zero coefficients and each $s_1$ coefficient is ≤5, the **total change from y to z is at most 300**. But y coefficients are ~500,000, so z appears completely random and doesn't reveal $s_1$. This is the **core security mechanism** of Dilithium.

# Signing: Steps 7-8 (Rejection Sampling and Hints)

## 7 Rejection Sampling

Verify z doesn't leak information about $s_1$. **Critical for security!**

**Check 1: Is z bounded?**

$||z||_\infty < \gamma_1 - \beta$

where $\beta = 60 \times \eta = 300$
So: $||z||_\infty < 523{,}476$

**Check 2: Are low bits small?**

$r_0 = \text{LowBits}(w - c \cdot s_2, 2\gamma_2)$

$||r_0||_\infty < \gamma_2 - \beta$

where $\gamma_2 - \beta = 261{,}588$

**If ANY check fails:** RESTART with new y

## 8 Create Hints h

Create **hints to help verifier** reconstruct $w_1$ without revealing $t_0$.

Computation:

$h = \text{MakeHint}(-c \cdot t_0, w - c \cdot s_2, 2\gamma_2)$

h: vector of bits (0 or 1 per coefficient)

$h_i = 1$ indicates "carry" at position i

At most $\omega = 96$ hints can be 1

How hints work:

Verifier computes: $A \cdot z - c \cdot t_1 \cdot 2^d$

$= w + c \cdot t_0 - c \cdot s_2$ (but doesn't know $c \cdot t_0$)

Using hint $h_i$:

$h_i = 0$: no carry, extract normally

$h_i = 1$: carry occurred, adjust by ±1

**Size:** At most 96 positions, ~96 bytes

## Expected Retry Statistics

| 4-7 | < 1% | ~99% |
|---|---|---|
| Average attempts | Failure probability after 20 attempts | Success probability per attempt |

# Signing: Step 9 and Final Signature

## 9 Final Validation

Perform **final checks** before outputting signature.

**Check 1: $c \cdot t_0$ bounded**

$\|c \cdot t_0\|_\infty < \gamma_2$

Ensures hidden part $c \cdot t_0$ is bounded
If fails → RESTART

**Check 2: Hint count valid**

Number of 1s in $h \leq \omega = 96$

At most 96 hint bits can be set
If more needed → RESTART (very rare)

**If ALL checks pass:** Output signature!

## Final Signature

$$\sigma = (z, h, c)$$

**z:** Vector of $\ell$=4 polynomials
256 coefficients each, ~20 bits each
≈ 2,560 bytes

**h:** Hint vector
At most 96 one-bits
≈ 101 bytes

**c:** Challenge polynomial
256 bits for positions + 60 bits for signs
≈ 40 bytes

**Total Size:** ~2,701 bytes

## Signature Size Breakdown

| 94.8% | 3.7% | 1.5% |
|---|---|---|
| z (main signature component) | h (hint vector) | c (challenge) |

MODULE 05

# Verification Process

Validating signatures using only the public key

# Verification: Steps 1-3 (Expand, Hash, Reconstruct)

## 1 Expand Matrix A

**Regenerate** the public matrix from seed ρ.

Computation:

$A = \text{ExpandA}(\rho)$

Same 5×4 matrix of polynomials

Deterministic: same seed → same matrix

Uses SHAKE-128 (same as key generation)

**Space saved:** Store 32B seed vs ~23KB matrix

## 2 Hash Message

Compute **same message digest** as signer.

Computation:

$tr = \text{CRH}(\rho \,||\, t_1)$

$m = \text{CRH}(tr \,||\, M)$

Same 384-bit digest m

Must match what signer computed

Uses SHAKE-256

Example:

M = "Transfer $100 from Alice to Bob"

m = 0x7A3F...9E21 (must match!)

**Same computation** as signing Step 1

## 3 Reconstruct $w_1'$

The **CORE of verification**. Reconstruct high bits of w.

Computation:

$w_1' = \text{UseHint}(h, A \cdot z - c \cdot t_1 \cdot 2^d, 2\gamma_2)$

Mathematical relationship:

$A \cdot z - c \cdot t_1 \cdot 2^d$

$= A \cdot (y + c \cdot s_1) - c \cdot t_1 \cdot 2^d$

$= A \cdot y + c \cdot A \cdot s_1 - c \cdot t_1 \cdot 2^d$

$= w + c \cdot (t - s_2) - c \cdot t_1 \cdot 2^d$

$= w + c \cdot t_0 - c \cdot s_2$

**Hints adjust** for missing $c \cdot t_0$

## Why This Works

The verifier doesn't know $t_0$ (it's secret), but can compute $A \cdot z - c \cdot t_1 \cdot 2d = w + c \cdot t_0 - c \cdot s_2$. This differs from w by the term $(c \cdot t_0 - c \cdot s_2)$. The hints h tell the verifier where "carries" occurred due to this difference, allowing correct reconstruction of $w_1$. If the signature is valid, **$w_1'$ will match the $w_1$ computed during signing**.

MODULE 05

# Verification: Steps 4-5 (Challenge and Final Checks)

## 4 Recompute Challenge $c'$

Hash the message and reconstructed $w_1'$ to create challenge.

Computation:

$c' = H(m \| w_1')$

H: SHAKE-256 hash function

Same computation as signing Step 5

Produces challenge with 60 ±1s

Key property:

If $w_1' = w_1$ (from signing)

Then $c' = c$ (signature's challenge)

This proves signature validity!

**Binds signature** to message and $w_1$

## 5 Final Checks

Verify **three conditions** to accept or reject signature.

**Check 1: z bounded**

$\|z\|_\infty < \gamma_1 - \beta$

All coefficients < 523,476

**Check 2: Challenge matches**

$c' = c$

Recomputed challenge equals signature's

**Check 3: Hints valid**

Number of 1s in $h \leq \omega = 96$

At most 96 hint bits set

**ALL pass:** ACCEPT

**ANY fails:** REJECT

## Why This Proves Authenticity

Only someone with **secret key ($s_1$, $s_2$)** can create z such that $A \cdot z - c \cdot t_1 \cdot 2^d$ relates correctly to original $w_1$. The challenge c binds the signature to the message (via hash). The masking y prevents z from revealing the secret keys. The hints h allow verification without revealing $t_0$. **All three checks must pass** for the signature to be valid.

MODULE 06

# Complete Worked Example

Toy parameter demonstration with simplified numbers

MODULE 06

# Worked Example: Setup and Key Generation

## Toy Parameters

| | | |
|---|---|---|
| $q$ | 97 | (small prime) |
| $n$ | 4 | (degree) |
| $k, \ell$ | 2, 2 | (2×2 matrix) |
| $\eta$ | 2 | (secrets in $[-2, 2]$) |
| $\gamma_1, \gamma_2$ | 40, 20 | (small bounds) |
| $\omega$ | 2 | (challenge weight) |

**Much smaller** than real Dilithium for clarity

## Key Generation

**Step 1:** Seed $\rho$ = 0xAB12

**Step 2:** Matrix A (2×2):

$a_{00} = 23 + 45X + 12X^2 + 67X^3$
$a_{01} = 34 + 56X + 78X^2 + 23X^3$
$a_{10} = 67 + 23X + 45X^2 + 12X^3$
$a_{11} = 89 + 34X + 67X^2 + 56X^3$

**Step 3:** Secret vectors:

$s_1 = [1-2X+0X^2+1X^3, -1+1X-2X^2+0X^3]$
$s_2 = [2+0X-1X^2+1X^3, 0-1X+2X^2-1X^3]$

**Step 4:** Compute $t = A \cdot s_1 + s_2$

## Polynomial Multiplication Example

Computing $a_{00} \cdot s_1[0] = (23 + 45X + 12X^2 + 67X^3) \cdot (1 - 2X + 0X^2 + 1X^3) \bmod (X^4+1)$, q=97:

$= 23 - 46X + 0X^2 + 23X^3 + 45X - 90X^2 + 0X^3 + 45X^4 + 12X^2 - 24X^3 + 0X^4 + 12X^5 + 67X^3 - 134X^4 + 0X^5 + 67X^6$

Using $X^4=-1$, $X^5=-X$, $X^6=-X^2$: $= 33 + 36X + 92X^2 + 66X^3 \pmod{97}$

# Worked Example: Signing Message "Hello"

## Steps 1-3

**Step 1:** Hash message

M = "Hello"
m = 0x3F7A (simplified)

**Step 2:** Generate y

$y[0] = 15 - 20X + 30X^2 + 10X^3$
$y[1] = 25 + 5X - 15X^2 + 20X^3$
(coefficients in [-39, 39])

**Step 3:** Compute w = A·y

$w[0] = 67 + 23X + 45X^2 + 12X^3$
$w[1] = 34 + 78X + 56X^2 + 23X^3$

## Steps 4-6

**Step 4:** Extract $w_1$

$w[0]_0 = 67 = 1 \cdot 40 + 27$
$w_1[0]_0 = 1$

**Step 5:** Create challenge c

$c = 1 + 0X + 0X^2 - 1X^3$
(2 non-zero coefficients)

**Step 6:** Compute z

$c \cdot s_1[0] = 0 - 3X + 1X^2 + 2X^3$
$z[0] = 15 - 23X + 31X^2 + 12X^3$

## Steps 7-8

**Step 7:** Rejection sampling

$\|z\|_\infty = \max(15, 23, 31, 12) = 31$
Bound: $\gamma_1 - \beta = 40 - 4 = 36$
$31 < 36$ ✓ PASS

**Step 8:** Create hints h

h = vector of bits
indicating carries

**Signature:** $\sigma = (z, h, c)$

## Observing the Security Mechanism

Notice how **y coefficients (15, −20, 30, 10)** are much larger than **$s_1$ coefficients (1, −2, 0, 1)**. The challenge c only modifies z by small amounts (at most 4 in this toy example), so z appears random and doesn't reveal the secret key. This is the **masking principle** in action!

# Worked Example: Verification and Result

## Verification Steps

**Step 1: Regenerate A from seed**

Same matrix as key generation

**Step 2: Hash message**

m = 0x3F7A (same as signing)

**Step 3: Compute $A \cdot z - c \cdot t_1 \cdot 2^d$**

Should equal $w + c \cdot t_0 - c \cdot s_2$

**Step 4: Reconstruct $w_1'$**

Use hints h to adjust

**Step 5: Recompute c'**

$c' = H(m \| w_1')$
Should produce: $c' = 1 + 0X + 0X^2 - 1X^3$

## Final Verification

**Check 1: z bounded**

$\|z\|_\infty = 31 < 36$ ✓

**Check 2: Challenge matches**

$c' = 1 + 0X + 0X^2 - 1X^3$
$c = 1 + 0X + 0X^2 - 1X^3$
$c' = c$ ✓

**Check 3: Hints valid**

Number of hints ≤ 2 ✓

**ALL CHECKS PASS**

**SIGNATURE ACCEPTED ✓**

## What We Learned

This complete example demonstrates the **end-to-end flow**: key generation creates public and secret keys, signing produces a signature that proves knowledge of the secret key without revealing it, and verification confirms the signature is valid using only public information. The mathematical relationships ensure that **only the legitimate signer** can create valid signatures.

MODULE 07

# Security Analysis

Understanding the mathematical hardness assumptions

MODULE 07

# Security Foundations: MLWE and MSIS Problems

## MLWE Problem

**Module Learning With Errors** is the primary security foundation.

Problem Statement:

Given $(A, t = A \cdot s_1 + s_2)$
Find $s_1$ and $s_2$

A: random matrix of polynomials
$s_1$, $s_2$: secret vectors with small coefficients

- ✓ **A is random:** No hidden structure to exploit
- ✓ **$s_1$, $s_2$ are small:** Coefficients ≤ 5 (noise)
- ✓ **Noise hides structure:** $s_2$ makes t appear random

**Security Level:** **~125 bits**

Quantum security (requires 2^125 operations)

## MSIS Problem

**Module Short Integer Solution** ensures signature unforgeability.

Problem Statement:

Given A
Find short z such that $A \cdot z = 0 \pmod q$

A: random matrix of polynomials
z: short vector (small coefficients)

- ✓ **Short solution required:** Small coefficients only
- ✓ **Many solutions exist:** But finding short one is hard
- ✓ **Lattice problem:** Finding short vectors in high dimensions

**Relevance:** Forging a signature requires solving a variant of MSIS

## Quantum Resistance

Unlike RSA and ECDSA, which are broken by Shor's algorithm on quantum computers, **MLWE and MSIS remain hard even for quantum computers**. The best known quantum algorithms (Grover's search) only provide a square-root speedup, which is accounted for in the security parameters. This makes lattice-based cryptography the leading candidate for post-quantum security.

MODULE 07

# Why Rejection Sampling is Critical

## Attack Without Rejection

Without rejection sampling, the signature **$z = y + c \cdot s_1$ leaks information** about $s_1$:

Attack scenario:

1. Collect many signatures: $z_1, z_2, \ldots, z_N$
2. Average them: $(1/N)\Sigma z_i$
3. Since $y_i$ is random: $(1/N)\Sigma y_i \to 0$
4. But $(1/N)\Sigma c_i \cdot s_1 = s_1 \cdot (1/N)\Sigma c_i$
5. Doesn't cancel out → reveals $s_1$!

**Result:** Secret key can be recovered!

## Protection With Rejection

Rejection sampling ensures **z appears random** and independent of $s_1$:

**Only accept z if bounded**

$\|z\|_\infty < \gamma_1 - \beta$ ensures z looks "random enough"

**Rejected samples hidden**

Attacker never sees rejected signatures

**Uniform distribution**

Accepted z has uniform distribution (independent of $s_1$)

**Result:** Averaging attack fails!

## The Mathematical Insight

Rejection sampling is based on the **rejection sampling lemma** from probability theory. By carefully choosing the bounds ($\gamma_1 - \beta$ and $\gamma_2 - \beta$), we ensure that the distribution of accepted z is statistically close to uniform, regardless of the secret key $s_1$. This means that **no information about $s_1$ leaks** from the signature, even to computationally unbounded adversaries.

MODULE 08

# Summary: Key Generation, Signing, and Verification

## Key Generation

1. Generate seed $\rho$
2. Expand A from $\rho$
3. Sample $s_1$, $s_2$ (small)
4. Compute $t = A \cdot s_1 + s_2$
5. Split t into $t_1$, $t_0$
6. Output: $pk=(\rho,t_1)$, $sk=(\rho,K,tr,s_1,s_2,t_0)$

**Key insight:** Store only high bits $t_1$ in public key

## Signing

1. Hash message → m
2. Generate masking y (large)
3. Compute $w = A \cdot y$
4. Extract $w_1 = \text{HighBits}(w)$
5. Challenge $c = H(m||w_1)$
6. Compute $z = y + c \cdot s_1$
7. Rejection sampling (checks)
8. Create hints h
9. Output: $\sigma = (z, h, c)$

**Key insight:** y hides $s_1$ in z

## Verification

1. Expand A from $\rho$
2. Hash message → m
3. Reconstruct $w_1'$ using hints
4. Recompute $c' = H(m||w_1')$
5. Check: $||z||$
6. Accept or Reject

**Key insight:** Only signer can create valid z

## Why Dilithium is Special

- Post-Quantum Secure
- No Gaussian Sampling
- Constant-Time
- Compact Keys

CONCLUSION

# CRYSTALS-Dilithium: Securing the Post-Quantum Future

CRYSTALS-Dilithium represents the **future of digital signatures** in a post-quantum world. Its elegant design combines mathematical rigor with practical efficiency, making it suitable for real-world deployment.

By understanding the three core operations—**key generation, signing, and verification**—you now have the foundation to implement, use, or audit Dilithium-based systems.

### Quantum Resistant

Based on hard lattice problems that remain secure against quantum computers

### Efficient

Fast signing and verification with compact keys and moderate signature sizes

### Practical

Easy to implement securely with constant-time operations and no Gaussian sampling